

SIMATS ENGINEERING

ASSESSMENT TOOL 1 – High (Coding Challenge)

COURSE CODE: CSA09 COURSE NAME: Programming in Java

COURSE OUTCOME COVERED

CO1: Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations

(BL3,BL4)

QUESTIONS: 10 Total Marks: 100

ANNEXURE A Coding Challenge

Code of Conduct:

I **CH. Venkata chandu/192424140** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: chandu

Q1) Library Book Management using Classes & Encapsulation)

```
Main.java
1- import java.util.Scanner;
2-
3- class Book {
4-     private int bookId;
5-     private String title;
6-     private String author;
7-     private double price;
8-     private boolean issued;
9-
10-     Book(int bookId, String title, String author, double price) {
11-         this.bookId = bookId;
12-         this.title = title;
13-         this.author = author;
14-         this.price = price;
15-         issued = false;
16-     }
17-
18-     int getBookId() {
19-         return bookId;
20-     }
21-
22-     void issueBook() {
23-         if (!issued) {
24-             issued = true;
25-         }
26-     }
27- }
28-
29- Scanner sc = new Scanner(System.in);
30-
31- while (true) {
32-     System.out.println("1.Display\n2.Issue\n3.Return\n4.Exit\n");
33-     int choice = sc.nextInt();
34-
35-     switch (choice) {
36-         case 1:
37-             System.out.println("101 Java James 500.0\n102 Python Guido 400.0");
38-             break;
39-         case 2:
40-             System.out.println("Enter Book ID:");
41-             int id = sc.nextInt();
42-             Book book = new Book(id, " ", " ", 0.0);
43-             book.issueBook();
44-             System.out.println("Book issued successfully");
45-             break;
46-         case 3:
47-             System.out.println("Enter Book ID:");
48-             int id = sc.nextInt();
49-             Book book = new Book(id, " ", " ", 0.0);
50-             System.out.println("Book returned successfully");
51-             break;
52-         case 4:
53-             System.exit(0);
54-     }
55- }
```

```
Output
1.Display
2.Issue
3.Return
4.Exit
1
101 Java James 500.0
102 Python Guido 400.0
2
Enter Book ID:101
Book issued successfully
3
Enter Book ID:101
Book returned successfully
4
Exit
```

Q2) Employee Payroll System using Inheritance

```
Main.java
1- class Employee {
2-     String name;
3-     int id;
4-
5-     Employee(String name, int id) {
6-         this.name = name;
7-         this.id = id;
8-     }
9-
10-     double calculateSalary() {
11-         return 0;
12-     }
13-
14-     void display() {
15-         System.out.println("Employee ID: " + id);
16-         System.out.println("Name: " + name);
17-     }
18- }
19-
20- class PermanentEmployee extends Employee {
21-     double monthlySalary;
22-
23-     PermanentEmployee(String name, int id, double monthlySalary) {
24-         super(name, id);
25-         this.monthlySalary = monthlySalary;
26-     }
27-
28-     double calculateSalary() {
29-         return monthlySalary;
30-     }
31- }
32-
33- Scanner sc = new Scanner(System.in);
34-
35- while (true) {
36-     System.out.println("Employee ID:");
37-     int id = sc.nextInt();
38-     System.out.println("Name:");
39-     String name = sc.next();
40-     System.out.println("Monthly Salary:");
41-     double salary = sc.nextDouble();
42-
43-     PermanentEmployee emp = new PermanentEmployee(name, id, salary);
44-     emp.display();
45-     System.out.println("Salary: " + emp.calculateSalary());
46-     System.out.println("==== Code Execution Successful =====");
47- }
```

```
Output
Employee ID: 101
Name: Chandu
Salary: 50000.0
Employee ID: 102
Name: Rahul
Salary: 30000.0
==== Code Execution Successful =====
```

Q3) Vehicle Rental System using Runtime Polymorphism

```
Main.java
1- class Vehicle {
2-     String vehicleName;
3-
4-     Vehicle(String name) {
5-         vehicleName = name;
6-     }
7-
8-     int calculateRent(int days) {
9-         return 0;
10-     }
11-
12-     void display() {
13-         System.out.println("Vehicle: " + vehicleName);
14-     }
15- }
16-
17- class Car extends Vehicle {
18-     Car(String name) {
19-         super(name);
20-     }
21-
22-     @Override
23-     int calculateRent(int days) {
24-         return days * 2000;
25-     }
26- }
27-
28- class Bike extends Vehicle {
29-     Bike(String name) {
30-         super(name);
31-     }
32-
33-     @Override
34-     int calculateRent(int days) {
35-         return days * 800;
36-     }
37- }
38-
39- class Bus extends Vehicle {
40-     Bus(String name) {
41-         super(name);
42-     }
43-
44-     @Override
45-     int calculateRent(int days) {
46-         return days * 5000;
47-     }
48- }
49-
50- Scanner sc = new Scanner(System.in);
51-
52- while (true) {
53-     System.out.println("Vehicle Name:");
54-     String name = sc.next();
55-     System.out.println("Days:");
56-     int days = sc.nextInt();
57-
58-     Vehicle vehicle = new Vehicle(name);
59-     vehicle.display();
60-     System.out.println("Rent: Rs. " + vehicle.calculateRent(days));
61-     System.out.println("==== Code Execution Successful =====");
62- }
```

```
Output
Vehicle: Toyota Car
Days: 5
Rent: Rs. 10000
Vehicle: Yamaha Bike
Days: 5
Rent: Rs. 4000
Vehicle: Volvo Bus
Days: 5
Rent: Rs. 25000
==== Code Execution Successful =====
```

Q4) Bank Account System using Data Hiding

Main.java	Output
<pre>3- class BankAccount { 4- private int accountNumber; 5- private String holderName; 6- private double balance; 7- BankAccount(int accountNumber, String holderName, double balance) { 8- 9- this.accountNumber = accountNumber; 10- this.holderName = holderName; 11- this.balance = balance; 12- } 13- void deposit(double amount) { 14- if(amount > 0) { 15- balance += amount; 16- System.out.println("Deposit successful"); 17- } 18- else { 19- System.out.println("Invalid deposit amount"); 20- } 21- } 22- void withdraw(double amount) { 23- 24- if(amount > 0 && amount <= balance) { 25- balance -= amount; 26- System.out.println("Withdrawal successful"); 27- } 28- else { 29- System.out.println("Invalid withdrawal"); 30- } 31- } 32-}</pre>	<pre>1.Deposit 2.Withdraw 3.Check Balance 4.Exit Enter choice: 1 Enter amount: 50000 Deposit successful 1.Deposit 2.Withdraw 3.Check Balance 4.Exit Enter choice: 2 Enter amount: 25000 Withdrawal successful 1.Deposit 2.Withdraw 3.Check Balance 4.Exit Enter choice: 3 Account Number: 1001 Holder Name: Chandu Balance: 30000.0 1.Deposit 2.Withdraw</pre>

Q5) University Admission System using Method Overloading

Main.java	Output
<pre>1- import java.util.Scanner; 2- class Admission { 3- // Undergraduate fee calculation 4- double calculateFee(double tuitionFee) { 5- return tuitionFee; 6- } 7- // Postgraduate fee calculation 8- double calculateFee(double tuitionFee, double researchFee) { 9- return tuitionFee + researchFee; 10- } 11- // Scholarship student fee calculation 12- double calculateFee(double tuitionFee, double scholarship, boolean student) { 13- return tuitionFee - scholarship; 14- } 15- } 16- public class Main { 17- public static void main(String args[]) { 18- Scanner sc = new Scanner(System.in); 19- 20- Admission admission = new Admission(); 21- System.out.println("1. Undergraduate"); 22- System.out.println("2. Postgraduate"); 23- System.out.println("3. Scholarship Student"); 24- System.out.print("Enter choice: "); 25- int choice = sc.nextInt(); 26- double fee; 27- switch(choice) { 28- case 1:</pre>	<pre>1. Undergraduate 2. Postgraduate 3. Scholarship Student Enter choice: 1 UG Fee: 50000.0 === Code Execution Successful ===</pre>

Q6) Shape Area Calculator using Abstract Class & Polymorphism

Main.java	Output
<pre>1- abstract class Shape { 2- 3- abstract double area(); 4- } 5- 6- class Circle extends Shape { 7- double radius; 8- Circle(double radius) { 9- this.radius = radius; 10- } 11- double area() { 12- return Math.PI * radius * radius; 13- } 14- } 15- class Rectangle extends Shape { 16- 17- double length; 18- double breadth; 19- 20- Rectangle(double length, double breadth) { 21- 22- this.length = length; 23- this.breadth = breadth; 24- } 25- 26- double area() {</pre>	<pre>Area: 78.53981633974483 Area: 50.0 Area: 24.0 === Code Execution Successful ===</pre>

Q7) Hospital Management System using Interfaces

Main.java

Share

Run

```
1 interface MedicalRecord {
2     void addRecord();
3
4     void displayRecord();
5 }
6
7
8 class Patient implements MedicalRecord {
9     String name;
10    int patientId;
11    String disease;
12
13    Patient(int id, String name, String disease) {
14        patientId = id;
15        this.name = name;
16        this.disease = disease;
17    }
18
19    public void addRecord() {
20        System.out.println("Patient record added");
21    }
22
23    public void displayRecord() {
24        System.out.println("Patient ID: " + patientId);
25        System.out.println("Patient Name: " + name);
26        System.out.println("Disease: " + disease);
27    }
28 }
```

Output

Patient record added
Patient ID: 101
Patient Name: Chandu
Disease: Fever

Doctor record added
Doctor ID: 201
Doctor Name: Dr. Kumar
Specialization: Cardiology

=== Code Execution Successful ===

Q8) Online Shopping Order System using Packages

Main.java

Share

Run

```
1 class Product {
2     int productId;
3     String productName;
4     double price;
5
6
7     Product(int id, String name, double price) {
8         productId = id;
9         productName = name;
10        this.price = price;
11    }
12
13
14
15
16    double getPrice() {
17        return price;
18    }
19
20
21
22    void displayProduct() {
23        System.out.println("Product ID: " + productId);
24        System.out.println("Product Name: " + productName);
25        System.out.println("Price: " + price);
26    }
27 }
28 }
```

Output

Order placed successfully
Product ID: 101
Product Name: Laptop
Price: 50000.0
Total Amount: 50000.0

=== Code Execution Successful ===

Q9) Student Result Analyzer using Arrays of Objects

```
Main.java
1 class Student {
2
3     int rollNo;
4     String name;
5     int marks1, marks2, marks3;
6
7
8     Student(int rollNo, String name, int m1, int m2, int m3) {
9
10        this.rollNo = rollNo;
11        this.name = name;
12        marks1 = m1;
13        marks2 = m2;
14        marks3 = m3;
15    }
16    int calculateTotal() {
17        return marks1 + marks2 + marks3;
18    }
19    double calculateAverage() {
20
21        return calculateTotal() / 3.0;
22    }
23    String calculateGrade() {
24
25        double avg = calculateAverage();
26
27        if(avg >= 90)
28            return "A";
29
30        else if(avg >= 75)
31            return "B";
32
33        else if(avg >= 50)
34            return "C";
35
36        else
37            return "Fail";
38    }
39    void display() {
40        System.out.println("Roll No: " + rollNo);
41        System.out.println("Name: " + name);
42        System.out.println("Total: " + calculateTotal());
43        System.out.println("Average: " + calculateAverage());
44        System.out.println("Grade: " + calculateGrade());
45        System.out.println("-----");
46    }
47 }
48 public class Main {
49
50     public static void main(String args[]) {
51         Student students[] = new Student[3];
52         students[0] = new Student(
53             101, "Chandu", 90, 85, 95);
54         students[1] = new Student(
55             102, "Rahul", 70, 75, 80);
56         students[2] = new Student(
57             103, "Arun", 50, 55, 60);
58         int max = 0;
59         Student topper = students[0];
60         for(Student s : students) {
61             s.display();
62             if(s.calculateTotal() > max) {
63                 max = s.calculateTotal();
64             }
65         }
66     }
67 }
```

Output

```
Order placed successfully
Product ID: 101
Product Name: Laptop
Price: 50000.0
Total Amount: 50000.0

=== Code Execution Successful ===
```

Q10) Smart Home Device Controller using Hierarchical Inheritance

```
Main.java
1 class Device {
2     String deviceName;
3     boolean status;
4     Device(String name) {
5         deviceName = name;
6         status = false;
7     }
8     void turnOn() {
9         status = true;
10        System.out.println(deviceName + " is ON");
11    }
12    void turnOff() {
13        status = false;
14        System.out.println(deviceName + " is OFF");
15    }
16    int powerConsumption() {
17        return 0;
18    }
19    void display() {
20        System.out.println("Device: " + deviceName);
21        System.out.println("Power Consumption: "
22            + powerConsumption() + " watts");
23    }
24 }
25 class Light extends Device {
26     Light(String name) {
27         super(name);
28     }
29     @Override
30     int powerConsumption() {
31         return 20;
32     }
33 }
34 class Fan extends Device {
35     Fan(String name) {
36         super(name);
37     }
38     @Override
39     int powerConsumption() {
40         return 60;
41     }
42 }
43 class AirConditioner extends Device {
44     AirConditioner(String name) {
45         super(name);
46     }
47     @Override
48     int powerConsumption() {
49         return 1500;
50     }
51 }
52 }
53 public class Main {
54     public static void main(String args[]) {
55         Device devices[] = new Device[3];
56         devices[0] = new Light("Living Room Light");
57         devices[1] = new Fan("Ceiling Fan");
58         devices[2] = new AirConditioner("AC");
59         for(Device d : devices) {
60             d.turnOn();
61             d.display();
62             d.turnOff();
63             System.out.println("-----");
64         }
65     }
66 }
```

Output

```
Living Room Light is ON
Device: Living Room Light
Power Consumption: 20 watts
Living Room Light is OFF
-----
Ceiling Fan is ON
Device: Ceiling Fan
Power Consumption: 60 watts
Ceiling Fan is OFF
-----
AC is ON
Device: AC
Power Consumption: 1500 watts
AC is OFF
-----

=== Code Execution Successful ===
```